

■ 2.5 Procedural Programming

■ 2.5.1 Procedures

When you build up calculations in *Mathematica*, you often need to string together sequences of “commands”. You can do this using *procedures*. A procedure is simply a sequence of *Mathematica* expressions, separated by semicolons. The expressions in a procedure are evaluated in order, with the final result being the value of the last expression.

Each element of the procedure is “executed” in turn. The final result is the value of the last expression.

```
In[1]:= r = (1+x)^2; r = Expand[r]; r - 1
```

```
Out[1]= 2 x + x^2
```

This defines the function *f* to be a procedure. You need to include the parentheses to make it clear that the whole procedure is to be the value of *f*.

```
In[2]:= f[x_] := ( t = (1 + x)^2; t = Expand[t] )
```

Evaluating the function executes the procedure.

```
In[3]:= f[a + b]
```

```
Out[3]= 1 + 2 a + a^2 + 2 b + 2 a b + b^2
```

A side effect of executing the procedure is to assign a value to the variable *t*.

```
In[4]:= t
```

```
Out[4]= 1 + 2 a + a^2 + 2 b + 2 a b + b^2
```

When you set up procedures in *Mathematica*, you often need to use “temporary” variables to store values of intermediate results. You will usually want to arrange for each of these variables to be “local” to a particular procedure. You want to make sure that variables in two different procedures are independent, even if they happen to have the same name. For example, if two procedures both have a variable named *t*, you need to make sure that the value of *t* in each procedure is independent, otherwise you cannot, for example, expect to be able to call one of the procedures from within the other one.

You can use the construction `Block[{x, y, ...}, procedure]` to specify that a list of variables is to be treated as “local” to a particular procedure. Each time the procedure is executed, the original value of each local variable is saved, and is restored when the execution of the procedure is over. This means that you can define whatever value you want within the procedure without affecting the value outside. As a result, you can safely “nest” one procedure inside another, and thereby use *Mathematica* as a “block-structured” programming language.

Web sample page from The *Mathematica* Book, First Edition, by Stephen Wolfram, published by Addison-Wesley Publishing Company (hardcover ISBN 0-201-19334-5; softcover ISBN 0-201-19330-2). To order *Mathematica* or this book contact Wolfram Research: info@wolfram.com; <http://www.wolfram.com/>; 1-800-441-6284.

© 1988 Wolfram Research, Inc. Permission is hereby granted for web users to make one paper copy of this page for their personal use. Further reproduction, or any copying of machine-readable files (including this one) to any server computer, is strictly prohibited.

In this definition, `u` is declared as a “local” variable.

```
In[5]:= g[x_] :=
      Block[ {u}, u = (1 + x)^2; u = Expand[u] ]
```

`g` gives the same results as `f` did above.

```
In[6]:= g[a + b]
Out[6]= 1 + 2 a + a^2 + 2 b + 2 a b + b^2
```

However, since `u` is declared to be “local”, the temporary value it gets during the computation of `g` is removed when the computation is finished.

```
In[7]:= u
Out[7]= u
```

<code>command; command; ...</code>	a procedure consisting of a sequence of commands
<code>Block[{x, y, ...}, procedure]</code>	declare the variables <code>x, y, ...</code> to be local to the procedure
<code>Block[{x = x0, y = y0, ...}, procedure]</code>	specify initial values for local variables

The basic structure of procedures.

For most purposes, the simple description of local variables given in this section will be quite adequate. The complete interaction between local variables, function arguments and so on in a symbolic language like *Mathematica* is necessarily, however, quite complicated. Section 2.5.10 discusses more of the details.